

SERVICIO NACIONAL DE APRENDIZAJE - SENA

Diseño y desarrollo de servicios web - caso

Servicio web de autenticación para Barbería Harry Estilos

Evidencia de aprendizaje GA7-220501096-AA5-EV01

Aprendices:

Hernán Salazar Ospina

Diego Caicedo Saldarriaga

José Arteta Angulo

Instructora: Verónica M. Muñoz

Programa de formación: Análisis y Desarrollo de Software

Proyecto formativo: Barbería Harry Estilos - Módulo de agendamiento

Ficha: 3336210

Colombia, 2026

Contenido

Introducción	3
Objetivos	4
Análisis de la evidencia.....	5
Diseño de la solución.....	5
Arquitectura general	5
Reutilización de componentes existentes	6
Especificación de los endpoints.....	7
Implementación.....	8
Base de datos	9
Herramientas de versionamiento.....	10
Conclusiones	11
Referencias.....	12

Introducción

El módulo de inicio de sesión y registro de Barbería Harry Estilos ya existe desde la evidencia GA7-220501096-AA3-EV01, construido sobre servlets, JSP y una tabla "usuarios" en MySQL. Hasta ahora, ese módulo solo podía consumirse desde el navegador, a través de los formularios propios del proyecto. Esta evidencia agrega una segunda puerta de entrada: un servicio web tipo API que recibe las mismas credenciales por HTTP y responde en texto plano, sin depender de una página renderizada.

El ejercicio tiene un valor puntual: no se trata de programar el registro y el inicio de sesión de nuevo, sino de exponer la lógica que ya funciona a través de un protocolo distinto. Por eso el diseño que se describe a continuación gira en torno a una decisión central, qué tanto del proyecto existente conviene reutilizar y qué tanto conviene aislar, para no arriesgar un módulo que ya está probado.

Objetivos

Objetivo general

Diseñar y codificar un servicio web REST que permita registrar usuarios e iniciar sesión mediante usuario y contraseña, reutilizando el módulo de autenticación ya construido en el proyecto Barbería Harry Estilos.

Objetivos específicos

- Analizar el requerimiento de la evidencia y su relación con el módulo de sesión existente del proyecto.
- Definir la arquitectura del servicio y justificar la decisión de mantenerlo como módulo independiente del despliegue web actual.
- Reutilizar los componentes de acceso a datos y validación ya probados en AA3, sin duplicar reglas de negocio.
- Especificar los endpoints del servicio, sus parámetros y las respuestas esperadas ante credenciales correctas e incorrectas.
- Documentar el código con comentarios y mantener el control de versiones en el repositorio compartido del equipo.

Análisis de la evidencia

Antes de codificar, conviene precisar qué pide exactamente el enunciado. El servicio debe permitir registrar un usuario y debe permitir iniciar sesión recibiendo usuario y contraseña. Ante credenciales correctas responde con un mensaje de autenticación satisfactoria; ante credenciales incorrectas, con un mensaje de error en la autenticación. El código debe llevar comentarios y el proyecto debe manejarse con una herramienta de versionamiento.

Barbería Harry Estilos ya tiene usuario, ya tiene inicio de sesión, ya tiene base de datos y ya tiene módulos codificados desde AA2 y AA3. El paso que falta, y el que atiende esta evidencia, es incorporar el servicio web al mismo proyecto: tomar la lógica que ya existe y exponerla mediante solicitudes HTTP, en lugar de construir un caso aislado desde cero.

Diseño de la solución

Arquitectura general

El proyecto principal se despliega como un archivo WAR sobre Apache Tomcat, con JSP y servlets mapeados por anotación. Spring Boot, en cambio, trae su propio servidor embebido y espera administrar él mismo el ciclo de arranque de la aplicación. Mezclar ambos dentro del mismo módulo Maven es posible, pero exige reconfigurar el empaquetado y probar de nuevo todo el flujo JSP para confirmar que nada se rompió. Frente a ese riesgo, se optó por un segundo módulo Maven, independiente del WAR, que corre en un puerto distinto y se conecta a la misma base de datos.

Tabla 1

Alternativas de arquitectura consideradas para el servicio web

Alternativa	Ventaja	Motivo del descarte o la elección
Integrar el API al WAR existente	Un solo despliegue, un solo puerto.	Obliga a mezclar Spring Boot con el servidor embebido de Tomcat externo y el mapeo por @WebServlet; el riesgo de afectar el módulo JSP ya probado no se justifica para esta evidencia.
Servicio nuevo y aislado, con datos en memoria	Cero dependencias del proyecto.	No cumple el objetivo real de la evidencia: la guía pide incorporar el servicio al proyecto que ya vienen trabajando, no construir un caso desconectado de él.
Módulo Spring Boot aparte, mismo esquema MySQL	Aísla el riesgo sin duplicar datos.	Elegida. Corre en el puerto 8081, no toca el WAR, y cualquier usuario creado desde un canal puede autenticarse desde el otro.

Reutilización de componentes existentes

Ninguna de las clases que resuelven el problema de negocio se reescribió. Se copiaron tal cual desde el módulo principal, porque residen en un proyecto Maven distinto y Java no permite compartir código fuente entre dos módulos sin declarar una dependencia formal entre ellos. La copia queda documentada en el propio código, con una nota que indica su origen y la razón de la duplicación.

Tabla 2

Componentes reutilizados del módulo de sesión (GA7-220501096-AA3)

Componente	Origen	Función en el servicio web
Usuario	Modelo de dominio, AA3-EV01	Representa un registro de la tabla "usuarios": id, nombre, correo, hash y sal de la contraseña, rol y estado activo.
UsuarioDAO	Acceso a datos, AA3-EV01	Ejecuta las consultas preparadas contra MySQL: registrar un usuario y buscarlo por correo.
PasswordUtil	Utilidad de seguridad, AA3-EV01	Genera la sal, calcula el hash SHA-256 de la contraseña y verifica una contraseña ingresada contra el hash guardado.

Componente	Origen	Función en el servicio web
RegistroValidator	Validación, AA3-EV02	Comprueba formato de correo y longitud máxima de los campos, igual que en el formulario web.
ConexionDB	Conexión JDBC, AA3-EV01	Abre la conexión hacia la base de datos "barberia_harry_estilos", la misma que usa el WAR.

Especificación de los endpoints

Los dos endpoints reciben los datos como parámetros de formulario, no como cuerpo JSON. Esa decisión sigue la forma en que se explicó el controlador en clase, y además simplifica la prueba desde Postman con una solicitud de tipo x-www-form-urlencoded, sin necesidad de armar un objeto JSON a mano.

Tabla 3
Endpoints del servicio web de autenticación

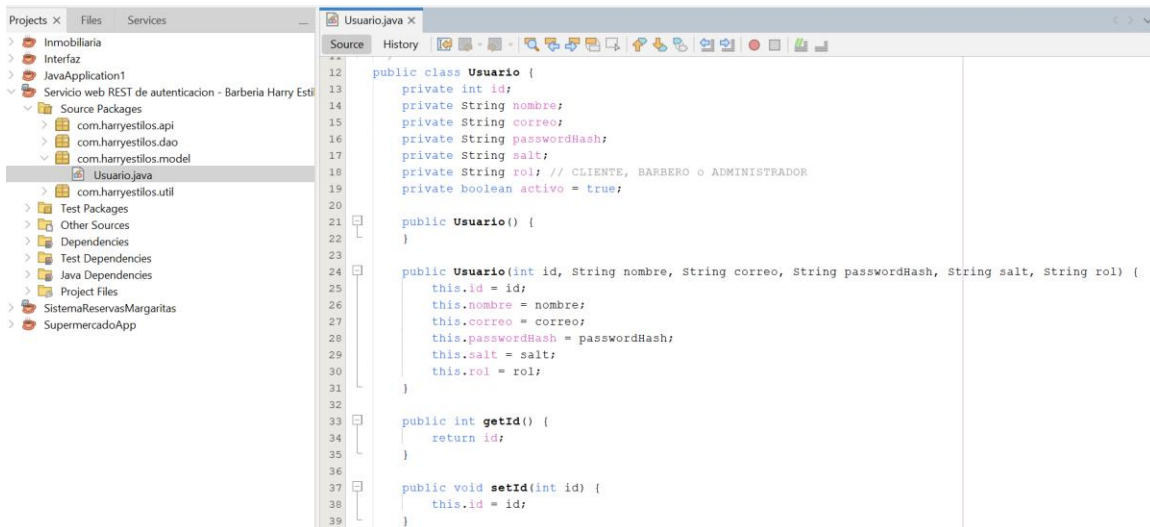
Endpoint	Parámetros	Respuestas
POST /api/usuario/login	usuario (correo), password	200: Autenticación satisfactoria. / 401: Error en la autenticación. / 403: cuenta inactiva. / 400: datos incompletos.
POST /api/usuario/registro	nombre, usuario (correo), password	201: Usuario registrado correctamente. / 409: el correo ya existe. / 400: datos incompletos o inválidos.

El campo "usuario" que pide el enunciado corresponde, en este proyecto, al correo electrónico: es el identificador real de inicio de sesión en la tabla "usuarios" desde AA3, y no existe una columna de nombre de usuario aparte. Se conservó el nombre "usuario" en el parámetro para no alejarse del enunciado, aunque internamente se use para consultar por correo.

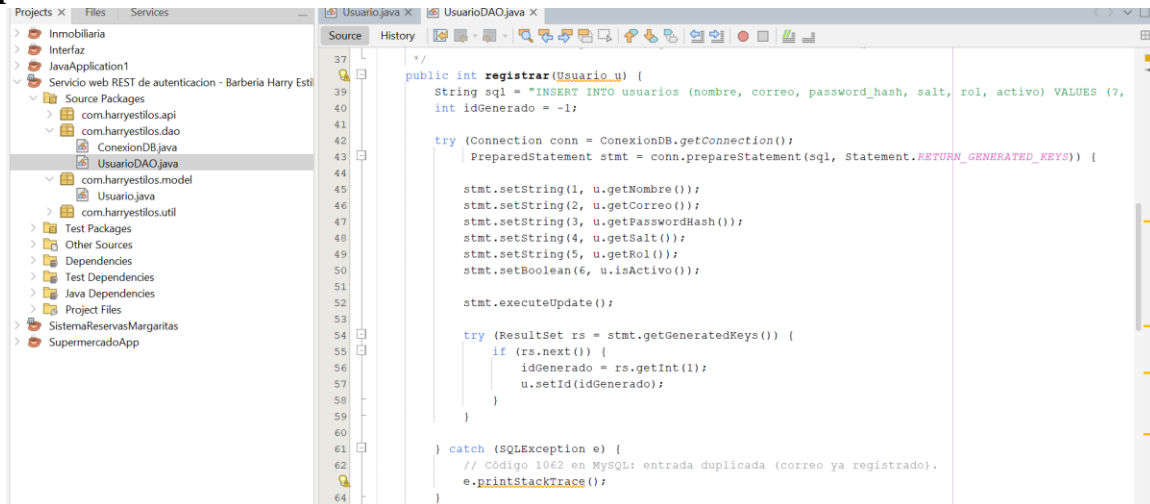
Implementación

A continuación se documentan las tres clases centrales del servicio: el modelo que representa al usuario, el DAO que lo persiste y el controlador que expone los dos endpoints, tal como quedaron codificados en el proyecto.

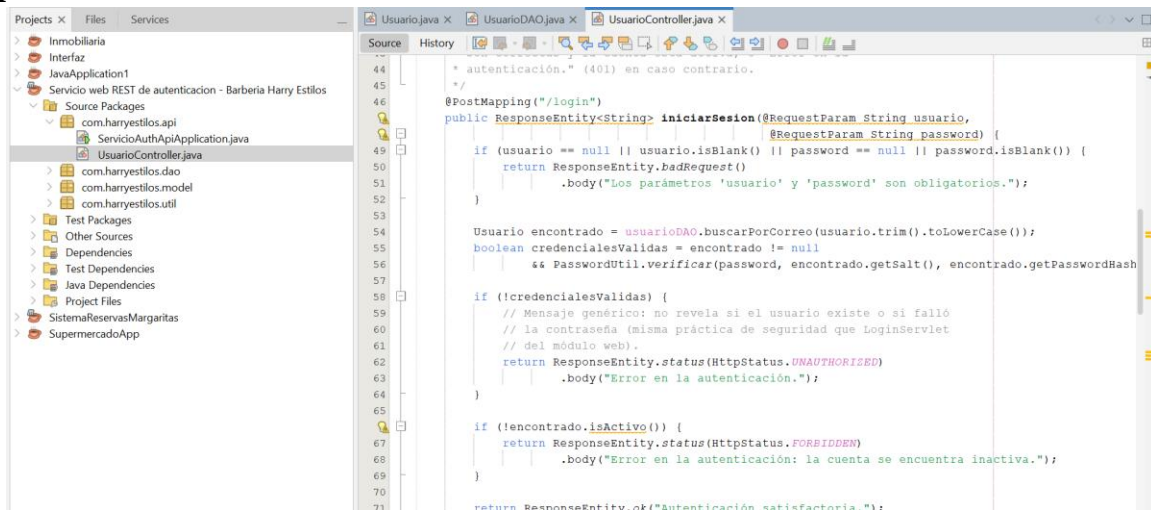
Captura 1: Clase modelo Usuario



Captura 2: Clase UsuarioDAO



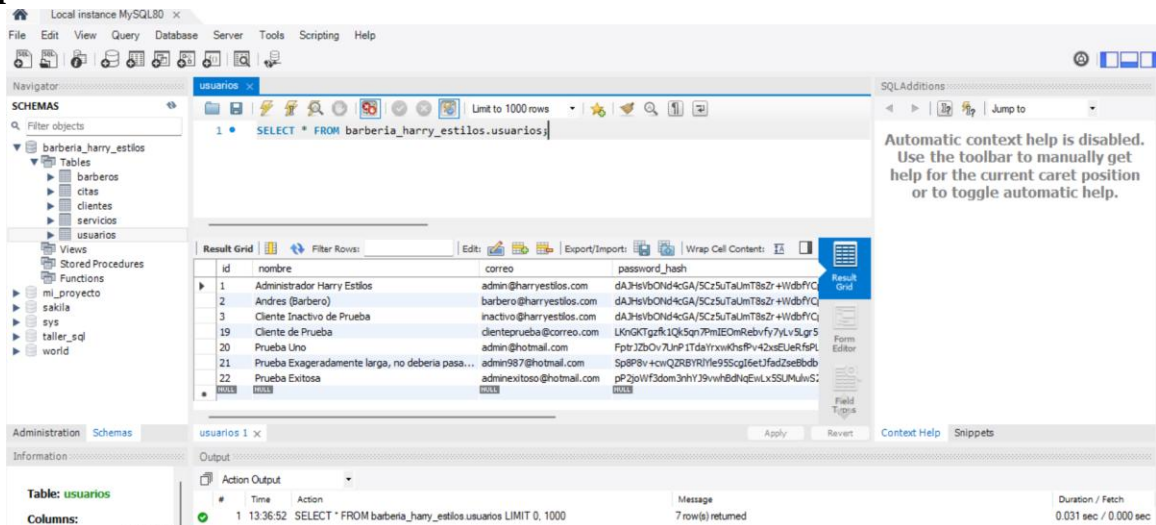
Captura 3: Clase UsuarioController



Base de datos

El servicio no crea tablas nuevas ni modifica el esquema existente. Usa la misma base de datos "barberia_harry_estilos" y la misma tabla "usuarios" que el módulo web del proyecto, para que un usuario registrado desde cualquiera de los dos canales pueda autenticarse desde el otro.

Captura 4: Base de datos



Herramientas de versionamiento

El servicio se agrega como una carpeta nueva, "servicio-auth-api", en la raíz del repositorio compartido del equipo en GitHub, junto al proyecto principal. El historial de commits de esa carpeta queda como evidencia del desarrollo, igual que en las entregas anteriores del proyecto.

Conclusiones

El servicio diseñado cumple el objetivo de la evidencia sin duplicar la lógica de autenticación que el equipo ya había construido y probado en AA3. Mantenerlo en un módulo aparte resultó ser la decisión que mejor equilibra dos necesidades que, en principio, compiten entre sí: incorporar el API al proyecto real, como pide el enunciado, y no arriesgar un despliegue JSP que ya funciona correctamente.

Queda pendiente, para la evidencia GA7-220501096-AA5-EV02, probar ambos endpoints con Postman y documentar los resultados con pantallazos, incluyendo al menos un caso de autenticación satisfactoria y un caso de error.

Referencias

Servicio Nacional de Aprendizaje [SENA]. (2024). Guía de aprendizaje 7 - GA7-220501096:

Codificar el software de acuerdo con el diseño establecido.

https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118_1/Contenido/DocArtic/GUI7/Guia_aprendizaje_7.pdf

Spring. (s.f.). Building a RESTful Web Service. Spring Framework.

<https://spring.io/guides/gs/rest-service>